Projet L3 : Cestionnaire C++

COURBET Nathan
WILBAUT Cristophe

Sommaire

Projet L3 : Cestionnaire C++.....1
1. Description et Objectif.....3
2. Pémisse du projet.....3
3. Cahier des charges.....4
3.1 Fonctionnalitées.....4
3.2 Problèmatiques.....4
4. Développement.....5
4.1 Outils envisagés (peut changer au cours du temps).....5
4.2 Base de l'application sans UI/UX.....5
4.3 Création du générateur de mot de passe.....6
4.4 Sauvgarde, modification, suppression.....7

1. Description et Objectif

Dans le cadre de ma troisième année de Licence Informatique, je réalise un projet intitulé « Cestionnaire » (jeux de mot avec gestionnaire et le langage C), c'est une application de gestion de mots de passe destinée à faciliter et sécuriser la gestion des identifiants personnels et professionnels des utilisateurs.

L'objectif principal de cette application est de créer, organiser, stocker et modifier les différents mots de passe d'un utilisateur à travers une interface graphique intuitive et ergonomique.

Le projet mettra l'accent sur la sécurité des données sensibles, en proposant deux modes de stockage :

- Localement, sur l'appareil de l'utilisateur
- À distance, sur une base de données sécurisée, selon les préférences et besoins de l'utilisateur.

L'application intégrera plusieurs mécanismes de protection tel que des algorithmes de chiffrements pour citer un exemple, qui seront mises en place afin de garantir la confidentialité et l'intégrité des informations.

À terme, "Cestionnaire" devrait se présenter comme une possibilitée sécurisée et conviviale de gestion des mots de passe, adaptée aussi bien aux particuliers qu'aux professionnels.

2. Pémisse du projet

Les raisons qui m'ont poussé à m'impliquer dans ce projet sont tout d'abord un besoin, la plupars des gestionnaires de mot de passe ne me donnaient pas envie ou bien étaient payant, j'ai donc eu l'idée de crée le mien.

Cela me procure deux avantages :

- Le savoir total sur comment les mots de passes sont géré dans mon application
- En apprendre beaucoup plus sur la cybersécurité en particulier la sécurité des données utilisateur

Ces différentes raisons m'ont donc poussé à crée mon propre gestionnaire de mot de passe.

3. Cahier des charges

3.1 Fonctionnalitées

- Génération de mots de passe sécurisé.
- Ajout, suppression, modification des entrées et mot de passe.
- Catégorisation des mots de passe (Tri par sites)
- Recherche et filtrage des informations.
- Le chiffrement des mots de passe et leur stockage

3.2 Problèmatiques

- Protection des mots de passes
- Gestion des sessions utilisateurs
- Respecter la RGPD
- Crée un interface ergonomique

4. Développement

4.1 Outils envisagés (peut changer au cours du temps)

Pour le dévloppement du « Cestionnaire » j'ai choisis différents outils qui me permettent une création plus organisée et efficace de l'application.

Nous pouvons noter en particulier,

- **Kate**, un IDE simpliste qui me permet de faire du c++ de manière agrèable.
- **Github** et **Git** pour avoir une gestion de versions et fichers.
- **Apache** un outil de creation de serveur.
- **Qt** un outil de création d'interface graphique.
- **MariaDB** une base de donnée pour le serveur.

4.2 Base de l'application sans UI/UX

Pour simplifier le développement de l'application, nous procédons par étapes. Cela ne sert à rien, et serait même contre-productif, de tout faire d'un coup.

Ma méthode de travail consiste donc à créer les fonctionnalités de base : la création, la modification et le stockage des différents mots de passe, sans se soucier de l'UI/UX (interface utilisateur).

L'application doit déjà être fonctionnelle dans le terminal avant d'ajouter une UI/UX. Cela permet de maintenir le code « propre », c'est-à-dire sans le code pour l'interface graphique, ce qui compliquerait énormément le développement, car le code serait beaucoup plus complexe.

Au lieu de ça, nous nous concentrons uniquement sur les fonctionnalités les plus importantes. L'UI/UX n'est nécessaire que pour simplifier l'utilisation.

4.3 Création du générateur de mot de passe

Nous commençons, à mon sens, par la partie la plus simple de ce projet : le générateur de mots de passe aléatoires.

Comme son nom l'indique, nous voulons que l'application crée un mot de passe en fonction des besoins de l'utilisateur.

```
void passGen(int *len, int *maj, int *num, int *spe){
    std::string mdp;

    std::string let = "abcdefghijklmnopqrstuvwxyz";
    std::string majLet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    std::string numb = "0123456789";
    std::string spec = "!@#$%^&*()-_=+[]{};:,.<>?/";

    if (*maj) let += majLet;
    if (*num) let += numb;
    if (*spe) let += spec;

    for(int i=0; i<*len; i++){
        mdp += let[rand() % let.size()];
    }
    std::cout << mdp << std::endl;
}
```

Sans trop rentrer dans les détails, cette fonction demande à l'utilisateur la longueur du mot de passe souhaitée et s'il veut ajouter des chiffres, des lettres majuscules ou des caractères spéciaux.

Ensuite, elle « pioche » aléatoirement dans les chaînes de caractères correspondantes et affiche le mot de passe généré dans le terminal.

Nous avons donc par exemple un résultat comme celui ci :

```
longeur du mdp : 8
voulez vous ajouter des majuscules ? (Y, n)Y
voulez vous ajouter des nombres ? (Y, n)Y
voulez vous ajouter des speciaux ? (Y, n)n


 votre mdp: g9myhJD7
```

Ici, le mot de passe contient bien les options demandées par l'utilisateur, c'est-à-dire des majuscules et des chiffres.

Le mot de passe obtenu est : **`g9myhJD7`**

4.4 Sauvgarde, modification, suppression

En ce qui concerne la partie principale du projet, c'est-à-dire la gestion de mots de passe, j'ai opté dans un premier temps pour le stockage local sous forme de JSON. Pour m'aider, j'utilise la bibliothèque JSON nommée « Json.h ».

À terme, je souhaiterais chiffrer le fichier JSON avec le mot de passe maître (Master) ; c'est-à-dire un mot de passe principal du compte. Pour l'utilisateur, il n'y aura donc qu'un seul mot de passe à retenir pour accéder à toutes les données.

Avant de faire ça, nous devons déjà pouvoir interagir avec le fichier JSON qui contient les mots de passe : la première étape consiste donc à sauvegarder une entrée.

La fonction se présente sous cette forme :

```
void ajouterEntree(const std::string& cheminFichier, std::string& site, std::string& username, std::string& password, std::string& note){
    std::ifstream input(cheminFichier);
    json data;

    if(input){
        input >> data;
        input.close();
    } else {
        data["entries"] = json::array();
    }

    json entree;
    entree["site"] = site;
    entree["username"] = username;
    entree["password"] = password;
    entree["note"] = note;

    data["entries"].push_back(entree);

    std::ofstream output(cheminFichier);
    output << data.dump(4);
    output.close();

    std::cout << "Mdp bien defini !" << std::endl;

}
```

Ici, nous récupérons différents champs : le mot de passe, évidemment, mais aussi le site web sur lequel il est utilisé, le nom d'utilisateur et une note.

```
entrez le nom du site : entuphf.fr
entrez votre username : nathan.courbet
entrez votre mdp : CeciEstUnBonMotDePasse2005!
entrez une note : ent
Mdp bien defini !
+----------------------------------------+
```

Nous créeons donc une nouvelle entrée pour tester si tout se déroule bien.

Ici, nous récupérons différents champs : le mot de passe, évidemment, mais aussi le site web sur lequel il est utilisé, le nom d'utilisateur et une note.


```
{
    "entries": [
        {
            "note": "",
            "password": "Test1234",
            "site": "google.com",
            "username": "usergoogle"
        },
        {
            "note": "un",
            "password": "password",
            "site": "huh.com",
            "username": "huh"
        },
        {
            "note": "ent",
            "password": "CeciEstUnBonMotDePasse2005!",
            "site": "entuphf.fr",
            "username": "nathan.courbet"
        }
    ]
}
```


Le projet s'arrête la pour l'instant, la création est en cours !